

# 3. Grundlagen

**Michael Schöttner**

Grundlagen Verteilter Systeme

SS 2024

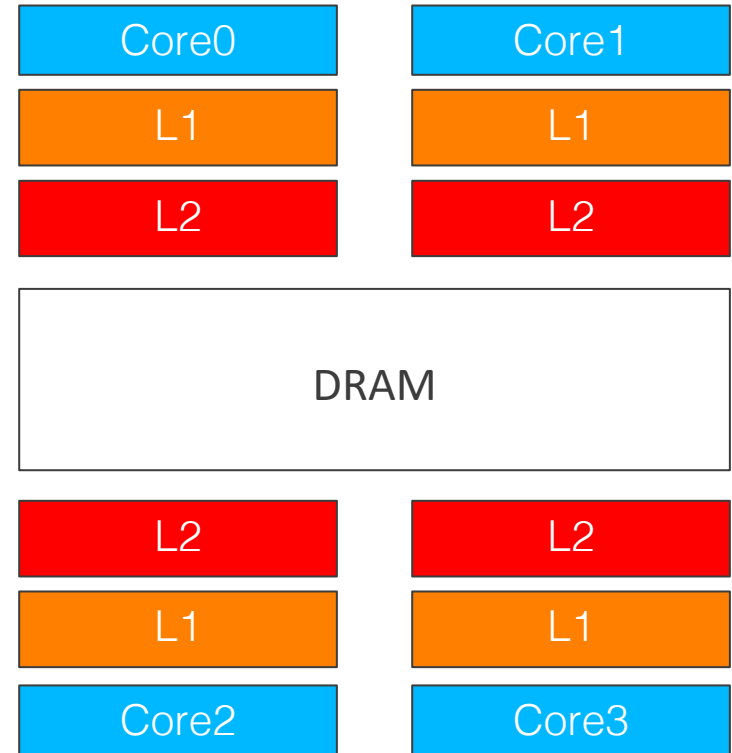
# 3.1 Vorschau

- Prozesse
- Threads in Java
- Sperren In Java
- Sichtbarkeit von Speicherzugriffen



# Speicherhierarchie

- Viele CPUs bestehen aus mehreren oder vielen Kernen (engl. cores)
- Jeder Kern hat eigene Caches
- Zugriffszeiten:
  - L1 Cache:  $\sim 1\text{ns}$
  - L2 Cache:  $3\text{ns}$
  - L3 Cache:  $15\text{ns}$
  - DRAM:  $\sim 70\text{ns}$
  - NVRAM:  $\sim 350\text{ns}$
  - SSD:  $\sim 150\mu\text{s}$



# Begriffe

- **Prozess:** Adressraum für einen oder mehrere Threads
- **Thread:** gehört zu einem Prozess
  - Kann auf alle Daten im Adressraum zugreifen
  - Alle Daten im Heap
  - Offene Dateien & Geräte
  - Hat aber einen eigenen Stack.
- Kooperierende Prozesse können auf einem oder verteilt auf mehreren Rechnern ablaufen



# Prozesse: fork () -Beispiel (Auszug)

```
/* process.c */
int main() {
    int pid; /* The pid of a child process. */

    printf("parent: pid=%d\n", (int)getpid());
    pid = fork();

    /* parent code */
    if ( pid ) { /* Value is non-zero in the parent. */
        printf("parent: data=%s\n", data);
    }

    /* child code */
    else {
        printf("    child: pid=%d\n", (int)getpid());
    }

    /* shared code */
    printf("finished\n");
}
```

gcc, Sprachstandard gnu99



# Pipes

- Bei einer parallelen Berechnung müssen die Kindprozesse ihre Ergebnisse an den Eltern-Prozess zurückmelden.
- Aufgrund der getrennten Adressräume geht das nicht direkt, sondern über Pipes (Standard in UNIX)
  - Zuverlässige Kommunikation
  - Ordnung der Bytes bleibt erhalten
  - Blockierung bei voller/leerer Pipe
  - Anonyme Pipe mit zwei Handles (read/write)
  - (Oder benannte Pipe Datei → Spezialdatei)



# Prozesse: Einfaches Pipe-Beispiel

```
/* simple_pipe.c */
int main() {
    char data[80];
    int  rb;                /* nr of read bytes */
    int  pipe_ends[2];      /* handles: read=0; write=1 */

    pipe(pipe_ends);        /* create anonymous pipe */
    if (fork()==0) {        /* child process */
        close(pipe_ends[1]); /* close write end -> child wants to read */
        rb = read(pipe_ends[0], data,79);
        data[rb]='\0';      /* terminate string */
        printf ("Received from parent: %s\n",data);
        close(pipe_ends[0]); /* done, close read end */
    }
    else {                  /* parent process */
        close(pipe_ends[0]); /* close read end -> parent wants to write */
        write(pipe_ends[1],"Hello World!",5);
        close(pipe_ends[1]); /* done, close write end */
    }
}
```

Elternprozess schickt einen String zum Kindprozess



# Threads

- Müssen explizit durch den Programmierer erzeugt werden
- Starten bei einer Funktion die der Programmierer definiert
- Somit können verschiedene Funktionen eines Programms nebenläufig oder simultan ausgeführt werden.
- Multicore-CPU's können mehrere Threads **simultan** ausführen, auch von verschiedenen Prozessen (→ separate TLBs pro Core)



# Threads: Erzeugung und Terminierung

- Jedes Java-Programm hat einen Haupt-Thread
- Erzeugen von Threads:
  - Interface `Runnable` implementieren oder Klasse `Thread` erweitern
  - Methode `run` ist Startpunkt des Threads, wird nicht von der Anwendung gerufen, sondern indirekt über `start`
- Terminierung von Threads:
  - Programm terminiert erst, wenn alle Threads terminiert sind
  - `tid.join` erlaubt es auf die Terminierung eines bestimmten Threads `tid` blockierend zu warten
  - Warten eines Threads ist mit `tid.interrupt` unterbrechbar



# Beispiel: 3 Threads die nacheinander ablaufen

```
public class ThreadExample implements Runnable {
    char thName;

    public ThreadExample (char name) { thName = name; }

    public void run () {
        for (int i=0; i<10; i++)
            System.out.print(thName);
        System.out.println();
    }

    public static void main( String[] args ) {
        Thread t1 = new Thread(new ThreadExample('A'), "T1");
        Thread t2 = new Thread(new ThreadExample('B'), "T2");
        Thread t3 = new Thread(new ThreadExample('C'), "T3");
        try {t1.start(); t1.join();
            t2.start(); t2.join();
            t3.start(); t3.join();
        } catch (InterruptedException e1) { }
    }
}
```

ThreadExample.java



# Threads: Wettlaufsituation

- Sobald mehrere Threads in einem Prozessadressraum ausführt werden kann es zu Wettlaufsituationen kommen
- **Wettlaufsituation** (engl. race condition): wenn nebenläufige Threads eines Prozesses auf gemeinsame Variablen schreiben, so ist das Ergebnis nicht deterministisch.
- **Lost update**: zum Beispiel wenn zwei Threads Werte auf eine gemeinsame Variable addieren  
→ Problem falls Core zw. Lesen, Addieren und Zurückschreiben entzogen wird



# Beispiel: Lost Update

```
public class Incrementer extends Thread {  
    static Incrementer lnx, rxz;  
    long count = 10000*10000;  
    static long asyn = 0;  
  
    public static void main( String[ ] s ){  
        lnx = new Incrementer();  
        rxz = new Incrementer();  
        lnx.start(); rxz.start();  
    }  
  
    public void run( ){  
        long zwisch;  
        while ( count-- > 0 ) {  
            zwisch = asyn+1;  
            asyn = zwisch;  
        }  
        System.out.println("asyn: " + asyn);  
    }  
}
```

Falls dazwischen die CPU entzogen wird, kann es zu Lost Updates kommen

Incrementer.java



## 3.4 Synchronisierung

- Wettlaufsituationen und Lost-Updates müssen mithilfe von Synchronisierung vermieden werden
- Hierzu verwendet man **kritische Abschnitte** (engl. critical regions)
  - = Programmabschnitt in dem sich nur ein Thread befinden
  - Ein- und Austritt in den kritischen Abschnitt wird synchronisiert
- Java bietet hierzu eine Reihe von **Sperren**, z.B. `synchronized`
  - Sperre funktioniert mithilfe von Objekten
  - Als Methodenattribut → Instanz oder Klassenobjekt (`static`) wird gesperrt
  - Oder angewandt auf eine Referenz: `synchronized(ref)` ;  
→ sperrt die durch `ref` referenzierte Instanz



# Beispiel: Lost Update mit Synchronisierung

```
public class IncrementerSynced extends Thread {  
    static Object mutex = new Object();  
    static IncrementerSynced lnx, rxz;  
    long count = 10000*10000;  
    static long syn = 0;  
  
    public static void main( String[ ] s ){  
        lnx = new IncrementerSynced();  
        rxz = new IncrementerSynced();  
        lnx.start(); rxz.start();  
    }  
  
    public void run( ){  
        long zwisch;  
        while ( count-- > 0 ) {  
            synchronized (mutex) {  
                zwisch = syn+1;  
                syn = zwisch;  
            }  
        }  
        System.out.println("syn: " + syn);  
    }  
}
```

CPU-Entzug an dieser Stelle ist immer noch möglich, aber kein Problem.

IncrementerSynced.java



# Signalisierung bei `synchronized`

- Manchmal muss man nach dem Belegen einer Sperre auf ein Ereignis warten
- Problematisch ist, wenn zum Auslösen dieses Ereignisses ebenfalls die gleiche Sperre erworben werden muss.
- Ohne weitere Synchronisierungskonzepte ist das Problem nicht lösbar
- Hierfür bietet Java, als auch andere Programmiersprachen, ein Signalisierungskonzept → `wait` und `notify`



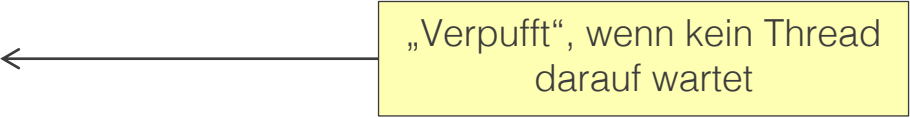
# Signalisierung bei `synchronized`

- `obj.wait`
  - Aufrufer gibt Sperre auf `obj` implizit frei und wird blockiert
  - Nur erlaubt, wenn eine Sperre auf `obj` gehalten wird
  - Optional auch mit Timeout
- `obj.notify`
  - Deblockiert einen Thread, der zuvor `obj.wait` gerufen hat
  - Letzterer läuft erst weiter, wenn der Aufrufer von `obj.notify` die Sperre auf `obj` freigegeben hat
  - Wartet kein Thread, so wird das Ereignis „notify“ verworfen. Es wird also nicht zwischengespeichert für später.



# Beispiel: Implementierung von Semaphoren

```
public class Semaphor {  
    private int count;  
  
    public synchronized void P() throws InterruptedException {  
        while (count <= 0) wait();  
        count--;  
    }  
  
    public synchronized void V() {  
        count++;  
        notify();  
    }  
  
    public Semaphor(int capacity) {  
        count = capacity;  
    }  
}
```



- Gibt es auch fertig in `java.util.concurrent.Semaphore`

Semaphor.java



# Beispiel: Anwendung der Semaphore

```
public class IncrementerSema extends Thread {  
    static Semaphore sema = new Semaphore(1);  
    static IncrementerSema lnx, rxz;  
    long count = 10000*10000;  
    static long syn = 0;  
  
    public static void main( String[ ] s ){  
        lnx = new IncrementerSema();  
        rxz = new IncrementerSema();  
        lnx.start(); rxz.start();  
    }  
  
    public void run( ){  
        long zwisch;  
        while ( count-- > 0 ) {  
            sema.P();  
            zwisch = syn+1;  
            syn = zwisch;  
            sema.V();  
        }  
        System.out.println("syn: " + syn);  
    }  
}
```

CPU-Entzug an dieser Stelle ist immer noch möglich, aber kein Problem.

IncrementerSema.java



# ReentrantReadWriteLock

- Manchmal ist ein wechselseitiger Ausschluss zu restriktiv
- Wenn beispielsweise mehrere lesende Threads parallel arbeiten dürfen oder nur einen Schreiber
- **Lösung: ReentrantReadWriteLock**
  - Besteht intern aus zwei Locks
  - `readLock().lock()` blockiert, falls eine Schreibe-Sperre vergeben ist und wartet auf Freigabe; mehrere Leser-Sperren sind erlaubt
  - `writeLock().lock()` blockiert, wenn eine Lese- oder eine Schreib-Sperre vergeben ist und wartet auf Freigabe
  - Optionale Fairness einstellbar über Konstruktor



# Beispiel

- 4 Threads die auf einem gemeinsamen String zugreifen
- 2 Threads geben den String auf der Konsole aus → Reader
- 2 Threads hängen konkurrierend einen Buchstaben an das String-Ende an  
→ Thread 1 immer „a“ und Thread 2 immer „b“.  
→ Writer
- Alle 4 Threads führen jeweils eine Schleife aus,  
wobei jeder Schleifendurchlauf synchronisiert wird.
- Der Zugriff auf den String (Klassenvariable) wird über eine Lese- und eine Schreib-Sperre synchronisiert.

# Beispiel: Code-Auszug

```
import java.util.concurrent.locks.ReentrantReadWriteLock;
```

```
public class ReentrantReadWriteLockExample {
```

```
    private static int NUM_INTERACTIONS = 10;
```

```
    private static String message = "";
```

```
    private static ReentrantReadWriteLock lock = new ReentrantReadWriteLock(true);
```

```
    static class Read implements Runnable {
```

```
        public void run() {
```

```
            for (int i=0; i<=NUM_INTERACTIONS; i++) {
```

```
                if (lock.isWriteLocked()) {
```

```
                    System.out.println("I'll take the lock from Write");
```

```
                }
```

```
                lock.readLock().lock();
```

```
                System.out.println(" ---> Message is " + message );
```

```
                lock.readLock().unlock();
```

```
            }
```

```
        }
```

```
    }
```

Fairness



Globale Var.



Benötigt keine Instanz  
von Outer-Class



Lesezugriff  
auf die  
globale  
Variable



ReentrantReadWriteLockExample.java



# Beispiel: Code-Auszug

```
static class Writer implements Runnable {  
    String ch;  
  
    public Writer(String ch) {  
        this.ch = ch;  
    }  
  
    public void run() {  
        for(int i = 0; i <= NUM_ITERATIONS; i++) {  
            try {  
                lock.writeLock().lock();  
                message = message.concat(ch);  
            } finally {  
                lock.writeLock().unlock();  
            }  
        }  
    }  
}
```

Schreibzugriff auf  
die globale Variable

ReentrantReadWriteLockExample.java



## 3.5 Java-Speichermodell

- Memory-Model auf HW-Ebene regelt wann Schreibzugriffe zwischen Cores und deren Caches sichtbar werden
- Das Memory-Modell von Java regelt drei Dinge
- **Atomicity**: Welche Operationen sind atomar, d.h. werden nicht durch andere Threads unterbrochen? → Sperren
- **Ordering**: In welcher Reihenfolge sehen Threads Schreibzugriffe? (Speichergriffe können zwischen verschiedenen Threads umsortiert werden)
- **Visibility**: Wann werden Modifikationen im Speicher anderen Threads sichtbar gemacht? Pro Thread und Core eigener Cache
- Siehe unter anderem: <http://www.angelikalanger.com/Articles/PDF/Java-Memory-Model.pdf>



# Visibility im Java-Speichermodell

- = Regeln wie Speicherzugriffe sichtbar werden bezogen, auf den thread-lokalen Cache
  - **Refresh**: thread-lokalen Cache aus dem Hauptspeicher befüllen
  - **Flush**: thread-lokalen Cache in den Hauptspeicher schreiben
- Threadstart und -ende:
  - Start: löst Refresh aus
  - Ende: löst einen Flush aus
    - nach einem `join` sehen wartende Threads alle Änderungen



# Visibility im Java-Speichermodell

- Synchronisierung (`synchronized` & andere Locks)
  - Erhalt einer Sperre → Refresh
  - Freigabe einer Sperre → Flush
  - Zwei Threads die sich über die gleiche Sperre synchronisieren sehen entsprechend gegenseitig alle Schreibzugriffe, die durch die Sperre synchronisiert wurden

# Visibility in Java

- Lese- und Schreibzugriff auf **volatile Variablen**
  - Lesen → Refresh
  - Schreiben → Flush
  - Erzwingt den Variablenzugriff immer über Hauptspeicher (nicht Cache)
  - Greifen zwei Threads auf eine volatile Variable zu, so bezieht sich die Sichtbarkeit nicht nur auf diese einzelne Variable, sondern auch den gesamten Cache jedes Threads → teuer
  - (Achtung: volatile Referenzen beschränken Verhalten auf Referenzen und erfassen nicht die Variablen der Instanz)



# Beispiel: NoVisibility.java

- Globale Variable wird verzögert vom Haupt-Thread gesetzt
- Zweiter Thread prüft diese Variable, sieht aber die Änderung des Haupt-Threads nicht
- Wird im zweiten Thread eine Ausgabe mit `System.out.print` eingefügt, so erfolgt eine implizite Synchronisierung beim Zugriff auf das Terminal, wodurch der Schreibzugriff sichtbar wird.

# Beispiel: NoVisibility.java

```
public class NoVisibility extends Thread {
```

```
    boolean keepRunning = true;
```

volatile löst das Problem

```
    public static void main(String[] args) throws InterruptedException {
```

```
        NoVisibility t = new NoVisibility();
```

```
        t.start();
```

```
        Thread.sleep(1000);
```

Warten, damit Änderung von keepRunning nach dem Start des 2. Threads erfolgt

```
        t.keepRunning = false;
```

```
        System.out.println("Main-Thread: keepRunning is now false");
```

```
    }
```

```
    public void run() {
```

```
        while (keepRunning) {
```

```
            //System.out.println("uncomment to solve visibility issue");
```

```
        }
```

```
        System.out.println("Second-Thread: finished");
```

```
    }
```

```
}
```

Ausgaben in Standard-Output werden synchronisiert.



# Zusammenfassung

- Programmierung von Multi-Threading ist fast immer notwendig, wenn verteilte Systeme gebaut werden
- Dies erfordert wiederum eine Synchronisierung der Threads
- Hierbei müssen auch folgende Aspekte beachtet werden
  - Sichtbarkeit von Speicherzugriffen

